RELEASE REPORT / 01 NOV 2026

CSTBS Version 3.0

Seat Reservations — features, data architecture, security, migration, and release notes

MILESTONE	SCOPE	COMPATIBILITY
v3.0	Seat booking	Extends V2

What changed in V3

• **Interactive seat map:** responsive 6 x 10 stadium grid, clear status legend, keyboard-accessible buttons, and a six-seat selection limit.

• **Near-real-time availability:** no-cache JSON endpoint; the browser refreshes occupancy every five seconds and updates the selection if a seat is taken.

• **End-to-end booking:** choose seats, review seat labels and total, confirm, and save multiple bookings in one database transaction.

• **My bookings:** every signed-in account can review active and cancelled booking history and cancel only its own reserved seats.

• **Model completion:** event-specific Seat inventory, per-seat Booking records, relationships, and stored price-at-booking snapshots.

• **V2 preservation:** password hashing, login/logout, student/staff/admin roles, public catalog, and admin event management are retained.

Version boundary

Reservations allocate seats, but they are not purchases. V3 intentionally excludes payments, ticket delivery, QR scanning, refund processing, and temporary seat holds. Availability refresh is polling, not WebSocket push.

Architecture & guarantees

Four persistent models

Model	V3 responsibility
User	Authenticated account with student, staff or admin role; owns bookings.
Event	Public event, admission price and scheduled time; owns event seats.
Seat	Named seat (Main / A-F / 1-10); unique per event, section and label.
Booking	One user + one seat + reserved/cancelled status + unit price snapshot.

Safe confirmation sequence

- Step 1 | Signed-in user selects one to six seats; client polls fresh occupancy.
- Step 2 | Review POST checks seat IDs, event ownership, limits, time and current availability.
- Step 3 | Confirm POST uses session-backed reviewed selection and repeats server checks.
- Step 4 | All Booking rows are committed as one transaction; SQLite rejects duplicate active seats. On a conflict the entire selection rolls back.
- Step 5 | Cancellation changes the owner's active booking to cancelled, preserving history and immediately releasing the seat.

Database-level double-booking protection

```
CREATE UNIQUE INDEX uq_active_booking_per_seat
ON bookings (seat_id) WHERE status = 'reserved';
```

This allows any number of cancelled history records while enforcing at most one active reservation per seat. The database constraint, rather than polling, resolves racing reservations.

HTTP endpoints introduced

Method / URL	Purpose
GET /events/<id>/seats	Interactive authenticated map
GET /api/events/<id>/seats	Public current occupancy JSON
POST /events/<id>/booking/review	Validate and display confirmation
POST /events/<id>/booking/confirm	Atomically save bookings
GET /bookings	Logged-in user history
POST /bookings/<id>/cancel	Owner-only cancellation (CSRF)

Migration, validation & GitHub release

Upgrading from Version 2

• **Back up** the V2 SQLite database and stop the old development server.

• Copy `CSTBS-v2/instance/cstbs.db` to `CSTBS-v3/instance/cstbs.db` before first launch.

• On startup, the local compatibility step adds `price_at_booking` if absent, initializes historical prices, and creates the unique active-seat index.

• V2 events with no seat rows get 60 default seats; any event with existing seats keeps its current seat map.

• Duplicate existing active bookings are rejected at startup; the migration will not silently delete historical rows.

Security, permissions & tests

• Flask-Login remains mandatory for selection, confirmation, history and cancellation. Admin CRUD is still admin-only.

• Changing requests use Flask-WTF CSRF protection. Cancellation looks up a booking by its ID **and** the current user ID.

• New tests cover multi-seat confirmation, stale review conflicts, CSRF, seat limits, cross-event seats, cancellation ownership, unique index, admin deletion rules and V2 schema upgrades.

• **Verification note:** Python files compile and Jinja2 templates parse. Live Flask integration tests could not run in this build environment because Flask dependencies were unavailable; run them locally before tagging.

Release identity

Item	Value
Recommended tag	v3.0
Release title	CSTBS v3.0 - Seat Reservations
Release summary	Interactive seat map, five-second availability refresh, atomic seat reservation, booking history, self-cancellation, and V2-preserving schema upgrade.

Run the project:

```
python -m pip install -r requirements.txt
python -m flask --app run.py seed-events
python run.py
python -m unittest discover -s tests -v
```

Local development only. No payment processing and no production-level scaling/security claim.